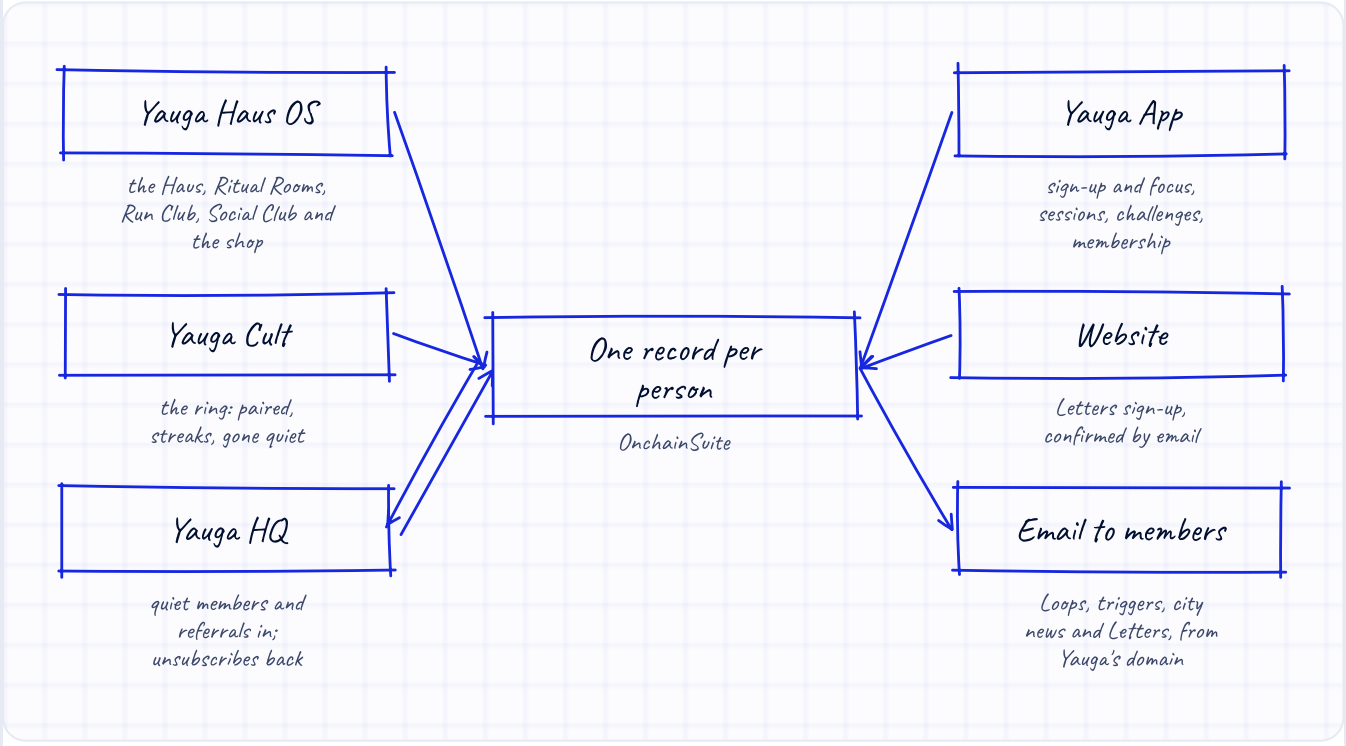
Yauga growth and retention strategy

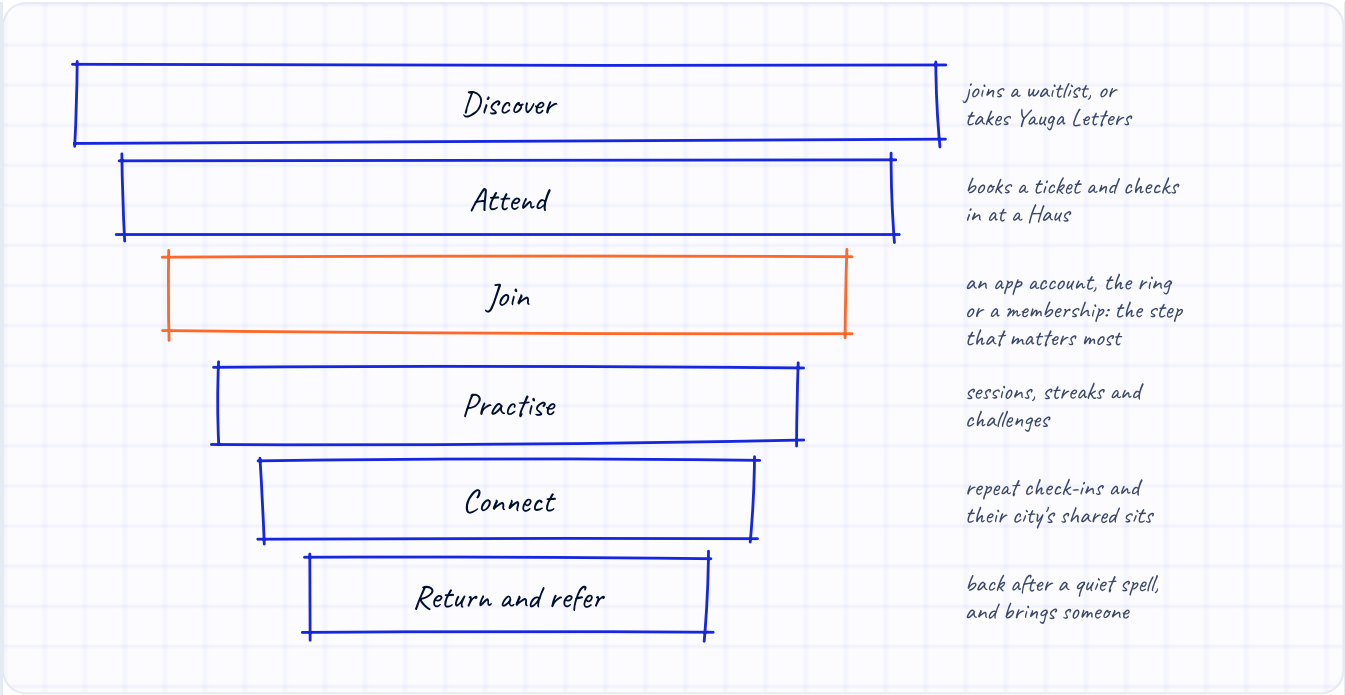
How email follows a member from a city's waitlist to the Haus, the app and the ring, and brings them back when they go quiet.

Today Yauga's systems do not know each other. Haus emails go out from Haus OS and Yauga Letters from OnchainSuite, and the app, Yauga Cult, the shop, Run Club and Social Club send nothing at all. A person who goes to a Haus and then joins the app is two strangers to the email system.

The proposal is one record per person. Every Yauga system reports what happened, and email follows the person rather than the product. The step that matters most is Attend to Join: live experiences bring people in, and email is how they find the app and the ring after they go home. The first automations we build sit on a check-in.

It has four parts: campaigns and the newsletter, Loops, lifecycle, and retention triggers. Email is the channel that carries them, from Yauga's own sending domain; the app keeps its own push, and Haus OS keeps its receipts. One broadcast a week, and every email passes the Yauga Test.





What we never send

Body data stays with Yauga. Sleep, heart rate, HRV, stress and SpO_2 readings never leave Yauga's systems. OnchainSuite only hears that something happened, such as a seven-day streak, never the readings behind it. That matches "your data is yours", and it keeps health data out of the email system altogether.

So we never send marketing to someone who has not said yes: a ticket buyer gets their ticket, and gets Yauga Letters only if they opted in, and every automation checks for the letters tag before it sends. We never send more than one broadcast a week, and since OnchainSuite has no built-in limit on how often one person is emailed, that rule is the limit. Nothing goes out that is not secular, and anything at 40 or above on the woo-woo scale is rewritten or cut. City leads never send their own emails, because that splits the voice and the sending reputation, and receipts never move out of Haus OS, because they work there today.

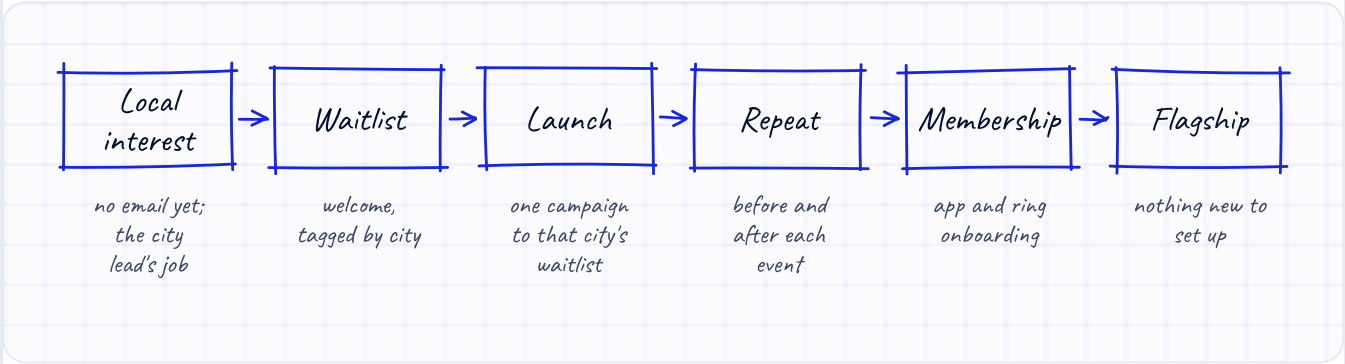
What we do send passes the Yauga Test: can it be felt, practised and explained plainly? Every moment has one owner, so nobody gets two emails about the same thing. Every member email comes from one sender with one standard in every city, and practitioners, city leads and partners sit on a separate list, never mixed into member sends.

Campaigns and the newsletter

A campaign goes to a tag, because something is happening for the people who carry it. When a city launches, one launch campaign goes to that city's waitlist. Before each event, a campaign goes to that event's ticket tag with what to bring and how to arrive ready, because an automation cannot wait

until two days before an event date; if Haus OS sends a reminder two days out, that event takes over. Local city news is written by the city lead and sent by HQ to the city tag, from the same sender as everything else, so the parts that must not drift stay the same in every city.

The same system runs every city, because a city is only a tag. One workspace holds every city, so a member who moves from Bournemouth to Barcelona stays one person with one history, and reporting never splits. Chicago and Vienna get separate sends in their own time, since date triggers run on one team time zone.



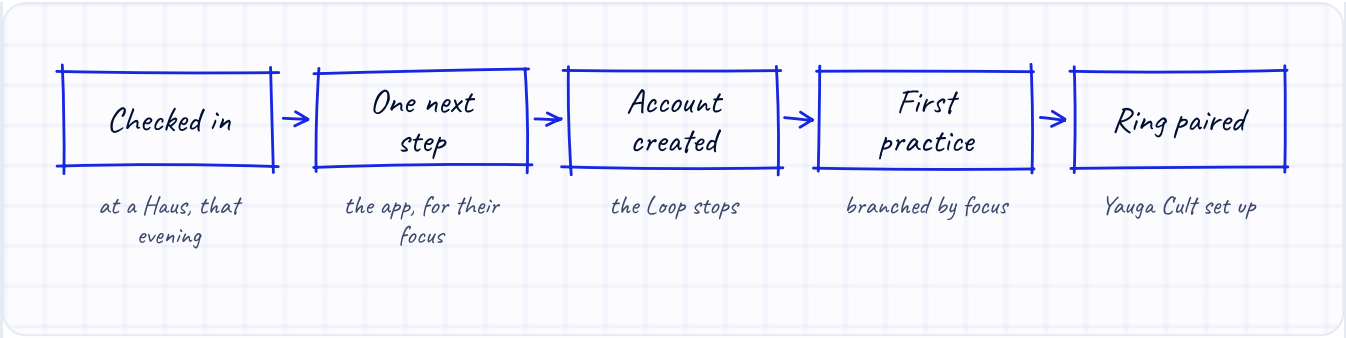
Yauga Letters is the one scheduled send: one email a week, as one broadcast split by focus and city. The member's pillar carries the practice, whether that is sleep and Tranquility, stress and Stillness, attention and Attention, or movement and Energy, and the city section carries what is on near them. It goes only to people with the letters tag. Selling to employers, universities and studios stays outside OnchainSuite, because that is sales, not member email, and members who join through them get a source tag so we know where they came from.

Loops

A Loop is a sequence of emails to one member. It starts from something they did, or did not do, and it stops the moment they take the step it asks for, so nobody is chased for something they already did. On OnchainSuite a member enters a "joined a segment" flow only once, ever, so every Loop here starts from a fresh event each time, and moments that repeat, each event and each quiet spell, start again from scratch.

The welcome Loop starts at capture. Someone who joins a city's waitlist or signs up for Yauga Letters on the website goes through an OnchainSuite form that asks them to confirm by email with one click. Once they confirm, one email says what they will get, Letters and news from their city, and points at the one next thing: their city's next event, or their place on the waitlist until launch. It stops on confirmation, and nothing else goes to an address that has not confirmed. Opt-ins at checkout are passed to the same form from Yauga's server.

The after-the-Haus Loop is the one that matters most, and it starts on a check-in. That evening, one email with one next step: the app, with a first practice for the focus they came for. It stops when they create an account. Live experiences bring people in; this is how they find the app after they go home.



The first-practice Loop starts when an account is created, branched by the focus they chose: a first practice for their pillar, then how to get the ring set up. It stops when they complete a session or pair the ring. When a streak is reached or a challenge completed, one email notices the progress and offers one small next practice; it is a single email per fresh event, so it fires again at the next milestone.

The slipping Loop starts when a member moves to At risk, which Yauga signals as member_quiet. It makes coming back easy: the next shared sit in their city, with the date, and nothing else. It stops when they check in or complete a session. At a third check-in a member is a regular, and one email invites them to bring someone; it stops when a referral is made.

The last Loop is one email. After 60 days without a check-in, a session or a streak, a Dormant member gets one good reason to come back. Then we leave them alone.

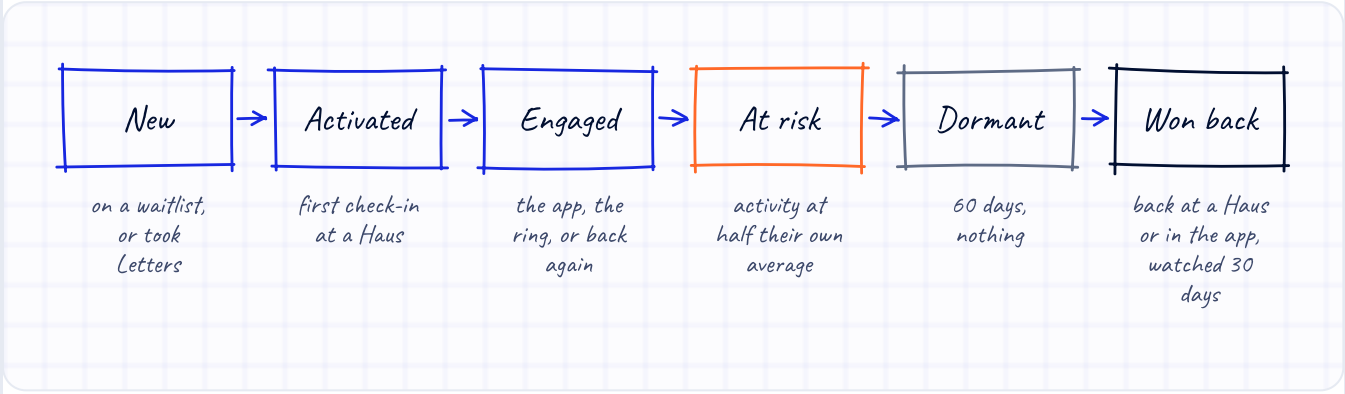
Email is the only channel for members. OnchainSuite's in-app messages and push reach people who sign in with a crypto wallet, which Yauga's members do not, and the Yauga App keeps its own push. Haus OS keeps ticket confirmations, receipts and refunds, which already work and must arrive instantly. One owner per moment, and one broadcast a week as the cap.

Lifecycle

Every member sits in one of six stages, the same six on every strategy we run: New, Activated, Engaged, At risk, Dormant and Won back. The stage is worked out from what the member did across every Yauga system, never set by hand, and it is read by city and by focus. The lifecycle moves on patterns in what a member does. Retention triggers, below, fire on single events, and the two never overlap.

A lifecycle starts at a capture point, and each capture point sets what we are allowed to send. The waitlist and Yauga Letters forms on the website carry news consent, confirmed by email. An opt-in tick at a Haus OS checkout carries the same, passed to the same form; a ticket bought without the tick carries service only, so that person gets their ticket and the follow-ups to the events they attend, and never Letters, launches or city news. An opt-in tick at app sign-up works the same way. The Haus emails Yauga holds today carry whatever those people originally agreed to, and where nobody can show what that was, the first email asks.

Everything hangs off one record per person, found by email first and Yauga member ID second, and labelled with five layers of tags as events arrive: who they are (member, attendee, waitlist, practitioner, city lead, partner), where (their city), their focus and its pillar, the formats they use (Haus, Ritual Room, Run Club, Social Club, the app, the ring, membership, Letters) and their state. Tags carry the meaning because OnchainSuite's segment builder reads tags, so a group like "Vienna, sleep focus" is always current without anyone rebuilding it. Groups that depend on history, such as "came to three Haus events but never opened the app", are built as a SQL segment from the event log.

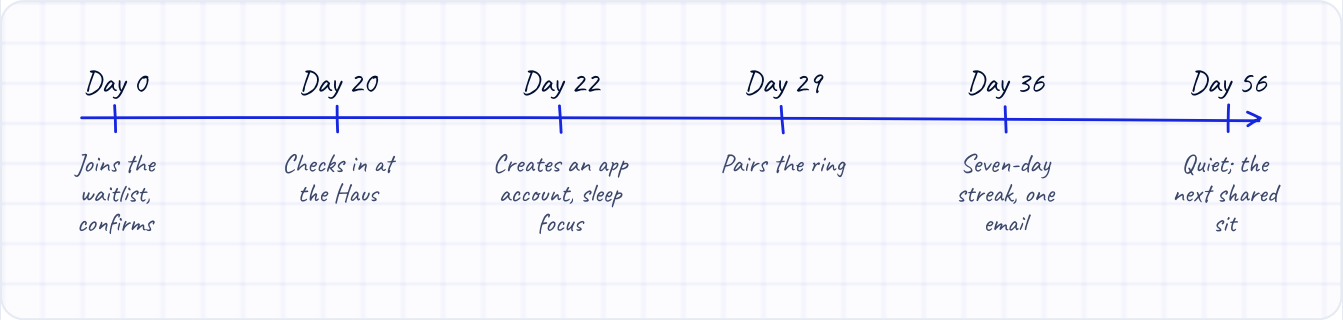


New is an email on a waitlist or taking Letters, with no check-in yet; the launch campaign and the welcome Loop move them on. Activated is a first check-in at a Haus; the after-the-Haus Loop moves them on. Engaged is a member who came back: an app account, a paired ring, a membership, or a second check-in, and no Loop runs for them: they get Letters and the city's campaigns. At risk is one rule on every strategy: activity at half the member's own 30-day average, counted across check-ins, sessions and streaks. OnchainSuite's built-in engagement scores are designed for wallets, so Yauga works this out from its own data and sends it as member_quiet, and the slipping Loop answers it. Dormant is 60 days with nothing, one email, then rest. Won back is a member who checked in or completed a session again from At risk or Dormant. They are watched for 30 days, then placed as Engaged or At risk like anyone else, and they never go back through the welcome.

A lifecycle also has exit points, and the rule that keeps them straight is that marketing consent can be withdrawn while the emails about a member's own activity continue. An unsubscribe stops Letters, launches, city news and every Loop, and it flows back to HQ Control by webhook the same day; tickets and receipts still come from Haus OS. A spam complaint stops everything that day, and a hard bounce closes the address. A membership cancelled is not an exit: it is a retention trigger, then the member is At risk or Dormant like anyone else. Rest itself is a state we count, so Dormant never looks like a backlog to work. An account deleted, or an erasure request, ends everything, the record included.

Take Ada. She joins the Vienna waitlist on the website and confirms with one click. When Vienna launches, the launch campaign reaches her and she books. Two days before, the ticket campaign tells her how to arrive ready. She checks in on day 20, and that evening one email points her at the app with a first practice for sleep. She creates an account on day 22, completes a first practice, and pairs the ring the following week. A seven-day streak earns one email that notices it and offers one

small next practice. In week eight she goes quiet, Yauga sends member_quiet, and she gets the next shared sit in Vienna, with the date. She checks in again.



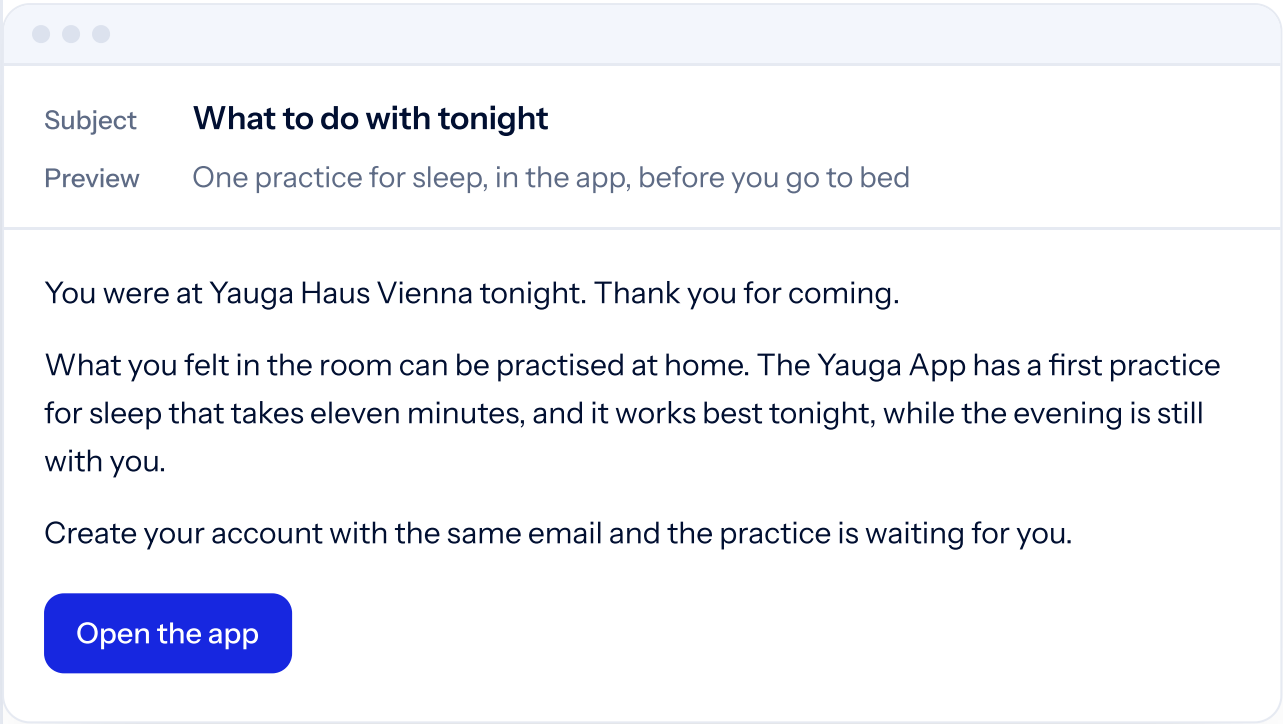
Retention triggers

A retention trigger is a single email sent the same day something happens to a member's own account. It comes from the event, never from a calendar, it carries no offer, and it goes to anyone the event belongs to, with or without the letters tag, because it is about something they did. It explains, then stops.

Four moments get one. A streak reached or a challenge completed: Yauga works out the streak and sends only the milestone, so the email notices it and offers one small next practice. The ring gone quiet: when Yauga Cult reports it inactive, one email says it has stopped syncing and how to get it going again. A membership cancelled: what changes, what stays, and how to come back, with no offer attached. And two days before an event, if Haus OS sends a reminder_due event; otherwise the ticket campaign carries that moment.

A refund gets nothing from OnchainSuite, because Haus OS sends the receipt and a receipt must arrive instantly. Here is the after-the-Haus email, the one every other automation depends on, written to pass the Yauga Test.

After a first check-in, email



How we know it worked

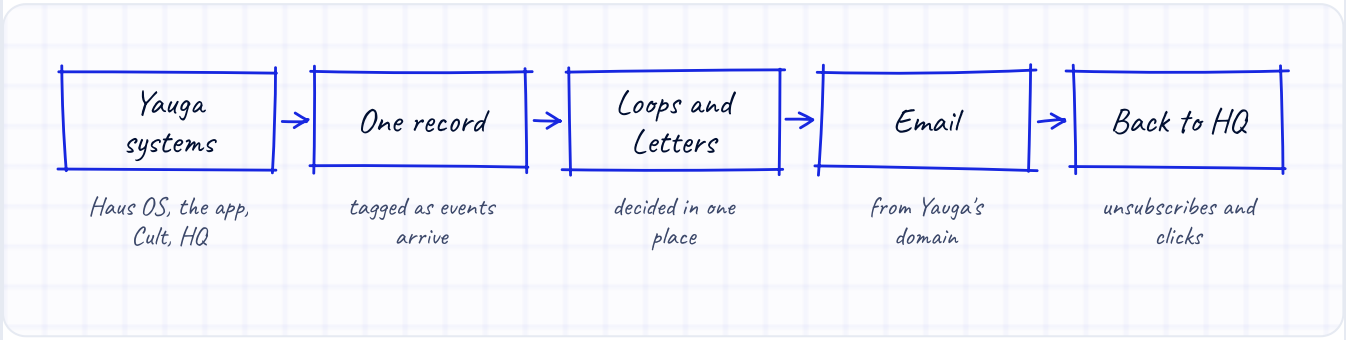
We measure movement round the loop. Opens are a health check; the goal is movement round the loop. Real outcomes, bookings, check-ins, accounts and sessions, are counted from Yauga's own events, because the conversion figures in OnchainSuite's automation reports are estimates. Every always-on Loop keeps a holdout, a small group left out on purpose, so we can show what the emails caused.

Every Monday, a stage report shows how many members sit at each stage, by city and by focus, and where most of them are stuck. That stage is where the next week's effort goes. Under it, five numbers: the share of a city's waitlist who book its first event, the share of first-time attendees who create an app account, the share of new app members who complete a first session, repeat bookings per member, and unsubscribes and spam complaints, which should stay low and flat. Targets are set once the first month gives us a baseline.

Running it on OnchainSuite

Every Yauga system sends its events directly to OnchainSuite from Yauga's server, with a secret key. Haus OS reports a waitlist joined, a ticket booked, a check-in, a refund and an order. The app reports an account created, a session completed, a challenge completed, and a membership started or cancelled. Yauga Cult reports the ring paired, a streak reached and the ring inactive. Yauga HQ reports a member gone quiet and a referral made, both worked out from Yauga's own data. The website's opt-ins go through an OnchainSuite form with double opt-in, and unsubscribes and clicks

go back to HQ Control by webhook. An event lands in seconds, so a follow-up can go the same day as a check-in, which a nightly file could never do. Each system is wired once; after that, new emails are built in OnchainSuite with no further developer work.



Six things the platform does today shaped this plan. A member enters a "joined a segment" flow once, ever, so repeating moments start from fresh events. An automation cannot wait until two days before an event date, so pre-event emails go as a campaign to the event's ticket tag, or Haus OS sends a reminder two days out. The segment builder reads tags, plus a score and a wallet, so meaning goes into tags at the source and history uses SQL segments. The built-in engagement and churn scores are designed for wallets, so "quiet" comes from Yauga as member_quiet. Date triggers run on one team time zone, so city sends are scheduled separately. And the conversion figures in automation reports are estimates, so outcomes are counted from Yauga's events.

We weighed the alternatives. Every system sending events directly is the choice, because it is live, each system stands on its own, and one going down does not stop the others; Haus OS collecting the app's and the ring's events and forwarding them is possible, if Yauga's developers prefer one pipe, at the cost of the app and ring depending on Haus OS. Nightly file exports arrive late, cannot start a flow at the right moment, and events older than seven days are refused, so a one-off import is for existing history only. A workspace per city would turn one person in two cities into two records; one newsletter for everyone would lose the pillar and the city that the tags give; moving receipts into OnchainSuite adds risk for no gain to the member; and in-app messages and push through OnchainSuite are not available to members without a wallet, which we revisit if the member data economy brings wallets.

The plan is sized to contacts: Launch covers 2,500 and Growth covers 25,000. OnchainSuite brings the event list and the tag scheme, the segments and automations built and tested, Yauga Letters structured by focus and city, reporting on the loop rather than on opens, and a team running it alongside Yauga's.

Disclosure: OnchainSuite is a related party to Surgence Labs. We say so here so the recommendation is judged on what the platform does, and you are free to test it against any alternative.

What we need from Yauga

Six things before anything is wired. One member ID shared by Haus OS and the app, because if they do not share one, email becomes the only way to join a person's record together. A developer to connect each system once. An opt-in tick at checkout and at app sign-up. City and focus values sent the same way from every system. HQ Control listening for unsubscribes. And the city leads' local news sent to HQ, never out from the city.

Then, before anything goes out: the event names above confirmed by Asif, since they are a working draft; the spelling of "Yauga Cult" confirmed with Kris; the domain the email sends from and who manages its DNS, because SPF, DKIM and DMARC decide between inbox and spam; and who approves each email, and whose voice it goes out in. This page is the strategy. Copy and timing come after Yauga agrees the approach.